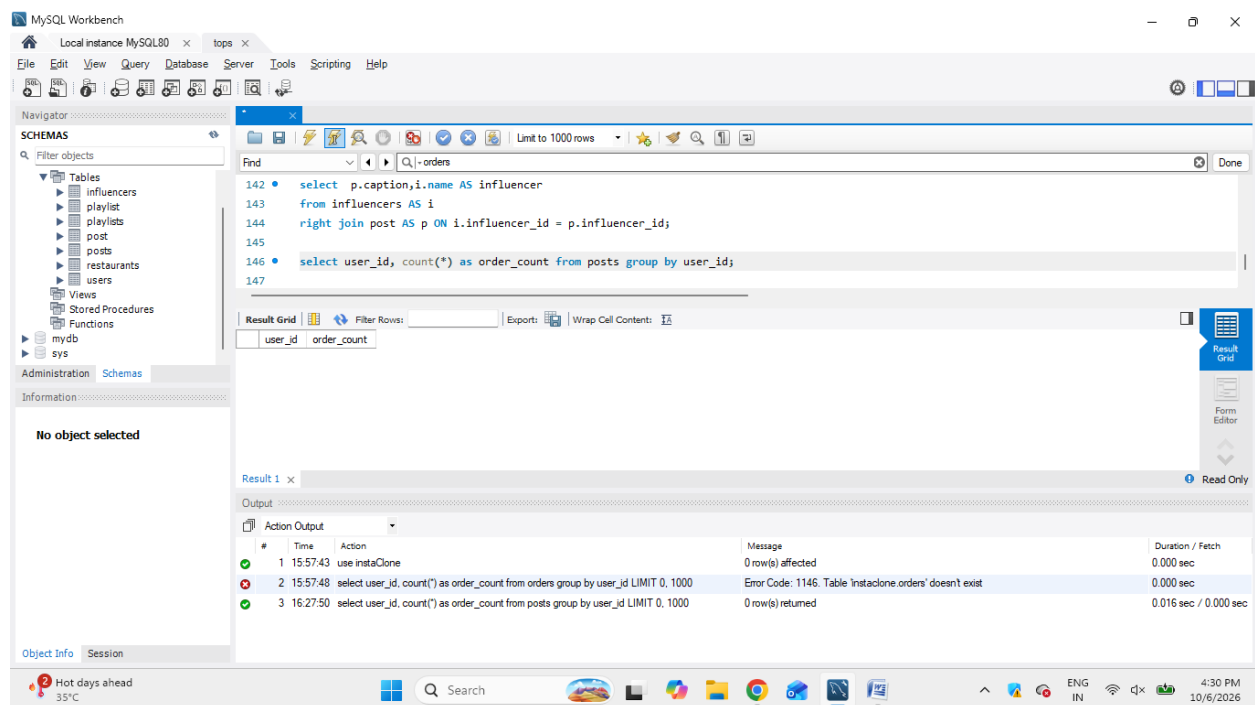


Session 6: Grouping and Subqueries
Trainer Coverage: GROUP BY, HAVING. Subqueries and nested queries. Example: “Show users whose average daily spend > ?500.”

Q:-1. Create a SQL query using GROUP BY to show the total number of orders placed by each user in an 'orders' table, displaying user_id and order_count.

select user_id, count(*) as order_count from posts group by user_id;



Q:-2 Write a SQL query to display the names of restaurants from a 'restaurants' table that have received an average rating above 4.0, using GROUP BY and HAVING.

select name, avg(rating) AS average_rating from restaurants group by name having avg(rating) > 4.0;

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'hellobd' selected. The main editor contains the following SQL code:

```

28 • insert into restaurants(name,cuisine,rating,city) values
29   ('Dominos', 'Pizza', 4.5, 'Rajkot'),
30   ('Honest', 'Gujarati', 4.2, 'Ahmedabad'),
31   ('McDonalds', 'Fast Food', 4.3, 'Rajkot'),
32   ('Barbeque Nation', 'North Indian', 4.6, 'Ahmedabad'),
33   ('The Grand Thakar', 'Gujarati', 4.4, 'Rajkot');
34
35 • select name, avg(rating) AS average_rating from restaurants group by name
36   having avg(rating) > 4.0;

```

The 'Result Grid' shows the output of the second query:

name	average_rating
Dominos	4.5
Honest	4.2
McDonalds	4.3
Barbeque Nation	4.6
The Grand Thakar	4.4

The 'Action Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	17:07:53	use hellobd	0 row(s) affected	0.000 sec
2	17:08:04	create table restaurants (id integer primary key auto_increment, name varchar(100), cuisine varchar(100), rating float(4), city varchar(100));	0 row(s) affected	0.063 sec
3	17:08:11	insert into restaurants(name,cuisine,rating,city) values ('Dominos', 'Pizza', 4.5, 'Rajkot'), ('Honest', 'Gujarati', 4.2, 'Ahmedabad'), ('McDonalds', 'Fast Food', 4.3, 'Rajkot'), ('Barbeque Nation', 'North Indian', 4.6, 'Ahmedabad'), ('The Grand Thakar', 'Gujarati', 4.4, 'Rajkot');	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.032 sec
4	17:08:18	select name, avg(rating) AS average_rating from restaurants group by name having avg(rating) > 4.0;	5 row(s) returned	0.000 sec / 0.000 sec

Q:-3. Suppose you have a 'payments' table with columns user_id, amount, and payment_date. Write a subquery to find user_ids who have made a single payment above ₹2000 in any transaction.

**select user_id from payments
where user_id in (select user_id from payments where amount > 2000);**

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'Schemas' tree with 'hellobd' selected. The main editor contains the following SQL code:

```

158 (1, 2500, '2026-10-02'),
159 (2, 1200, '2026-10-05'),
160 (3, 3000, '2026-10-04');
161
162
163 • select user_id from payments
164   where user_id in (select user_id from payments where amount > 2000);

```

The 'Result Grid' shows the output of the query:

user_id
1
3

The 'Action Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
3	18:05:20	create table payments (payment_id INTEGER PRIMARY KEY AUTO_INCREMENT, user_id INTEGER, amount DECIMAL(10,2), payment_date DATE);	0 row(s) affected	0.094 sec
4	18:09:43	select distinct user_id from payments where user_id in (select user_id from payments where amount > 2000);	0 row(s) returned	0.016 sec / 0.000 sec
5	18:10:27	select user_id from payments where user_id in (select user_id from payments where amount > 2000);	0 row(s) returned	0.000 sec / 0.000 sec
6	18:11:24	insert into payments (user_id, amount, payment_date) VALUES (1, 2500, '2026-10-02'), (2, 1200, '2026-10-05'), (3, 3000, '2026-10-04');	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.047 sec
7	18:11:34	select user_id from payments where user_id in (select user_id from payments where amount > 2000);	2 row(s) returned	0.000 sec / 0.000 sec

Q:-4. Using a nested subquery, display the names of movies from a 'movies' table that have a higher average rating than the overall average rating of all movies.
Hint: Use a subquery to calculate the overall average rating, then compare each movie's average rating against it in the main query.

```
select movie_name, avg(rating) as average_rating
from movies group by movie_name having avg(rating) > (select avg(rating)
from movies);
```

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'instacore' selected. The main editor window contains the following SQL query:

```
173 ('Movie A', 4.7),
174 ('Movie B', 3.5),
175 ('Movie B', 3.8),
176 ('Movie C', 4.8),
177 ('Movie C', 4.6);
178
179 select movie_name, avg(rating) as average_rating
180 from movies group by movie_name having avg(rating) > (select avg(rating) from movies);
```

The 'Result Grid' shows the following data:

movie_name	average_rating
Movie A	4.6
Movie C	4.6999999999999999

The 'Output' pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
1	18:19:05	use instaClone	0 row(s) affected	0.000 sec
2	18:19:10	create table movies (movie_id integer primary key auto_increment, movie_name text, ...	0 row(s) affected	0.031 sec
3	18:20:09	insert into movies (movie_name,rating) VALUES ('Movie A', 4.5), ('Movie A', 4.7), ('Movie ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.000 sec
4	18:22:07	select movie_name, avg(rating) as average_rating from movies group by movie_name havi...	2 row(s) returned	0.000 sec / 0.000 sec